

Scholo Education Assistant

Project Requirements - Phase 1 Revision

5 October 2026 | Authoritative implementation scope

Decision: Electron desktop shell

Electron replaces Tauri. React, local FastAPI, SQLite, and app-managed file storage remain the architecture baseline. The earlier Tauri and Rust shell decisions in the preserved historical requirements are superseded. The earlier shell memory target must be remeasured for Electron.

Decision: Focused Phase 1

Build Notes, local OCR, learning records, personal style profiles, and video citations with exact timestamps. Use one global activation shortcut for pointing or selecting screen content, followed by a comment/instruction popup. The student chooses a workspace folder containing organized subfolders, Markdown notes, JSON metadata, and assets. Video citations belong to notes and records and support exact moments or ranges. This scope replaces the broader Tier 1 delivery commitment for the initial release.

Delivery boundaries

The implementation baseline on the following pages describes the included workflows, staged enhancements, deferred modules, architecture, and completion checks. Tutor/RAG, models, mascot, labs, cloud routing, and portal automation remain future work. The single screen-capture activation is included. Video URLs and exact timestamps must be confirmed whenever player metadata is unavailable; automatic transcription is deferred.

Document precedence

This revision and its implementation baseline take precedence over conflicting statements in the original 2 October 2026 document, preserved at the end for traceability. Requirements describe intended behavior, not implementation status. Casrion is a reference for folder-based note taking; its README and license have been reviewed, and no code has been copied.

Phase 1: Notes, OCR, learning records, style profiles, and video citations

Scope decision

Build in Education-Assistant. Casrion (<https://github.com/DyneStein/Casrion>) is a reference for folder-based notes and background capture. Its README and GPL-3.0 license have been reviewed; no code has been copied from it.

The source requirements are the Scholo Education Assistant Project Requirements Document dated 2 October 2026. The user's narrower scope takes precedence over the document's complete Tier 1 scope. This document is a proposed implementation baseline; it does not claim the features are implemented.

The user also explicitly requested video citations with exact timings. Include this within notes and learning records. The user identifies it as a new feature relative to Casrion; that comparison has not been independently verified.

Included workflows

Single activation and folder workspace (user clarification, 5 October 2026)

- The student chooses a workspace folder. Notes, captured assets, OCR results, source references, comments, and later learning records/style profiles live inside that folder. Ordinary Markdown and JSON files remain readable without the app; SQLite is a local search/index aid.
- One global shortcut, Ctrl+Shift+Space, activates a screen overlay. No separate copy-text, screenshot, or quick-note shortcuts are registered. The student points to a target or drags a region, then receives a compact comment/instruction popup.
- Text is extracted locally with review before saving; images are retained as original capture evidence. Videos are stored as source citations with exact start/end times, not downloaded or copied as video content.
- The popup previews the capture, proposed content type, source, filing destination, and comment/instruction. The student can correct each item before saving. Never invent a video URL or timing when screen/accessibility metadata cannot provide it.
- Comments and instructions are retained with the captured data. Only implemented actions are executed; model-dependent explanations or summaries must not be presented as completed without a model.
- Create course/topic subfolders, Markdown notes, JSON metadata, and asset folders inside the selected workspace. All generated paths are constrained to that workspace. Keep runtime settings outside it only to remember the chosen folder.
- The desktop launch shortcut opens the app; closing the main window leaves the single capture shortcut available through the tray. Tray Quit exits the app and releases the shortcut.

Notes (NOT-1 through NOT-3)

- Create and edit text notes; import an image or screenshot as an attachment.
- Add a title, comment, course, topic, type, and tags. Show the destination before saving and allow corrections.
- Retain the original image, OCR text, source application, source timestamp, and optional file/page or URL reference. Mark unknown source information as unknown rather than inventing it.
- Suggest filing destinations from previous user-confirmed choices, with manual selection always available. Begin with deterministic rules; automatic AI classification is a later enhancement.

- Search and filter notes and export them as Markdown with source references and attached files.

OCR

- Extract text from imported images using a local OCR provider. Tesseract is the proposed first provider, pending a small accuracy evaluation; keep the provider replaceable.
- Preserve raw OCR output separately from student-corrected text and record provider/language details.
- Show queued, processing, succeeded, and failed states, with retry and understandable errors. OCR failure must not prevent saving the original note.
- Run work outside the interactive request path. Bound image dimensions, upload size, and job concurrency; validate image contents before processing.
- Core acceptance covers English printed text. Handwriting and additional languages require separate measured support.

Video citations with exact timings

- Attach one or more citations to a note or learning record: video URL or imported local video, title, optional creator, start time, optional end time, and the student's comment.
- Accept readable timestamps such as 01:23 and 01:02:03.250; store integer milliseconds to preserve precision. Reject malformed or negative times and end times before the start. Validate against duration when known.
- Keep the original URL and exact timing as canonical evidence. Generate a provider-supported link to the cited moment where possible; always display the exact timestamp or range even when the provider rounds playback precision or does not support deep links.
- Export title, source, timing, comment, and link together so a reference still makes sense outside the app. Imported local videos use portable relative references in export bundles and app-owned copies for persistence.
- Support multiple cited moments from the same video. Offline users can read all saved citation metadata and play imported local videos; remote video playback requires connectivity.
- Allow an optional manually supplied transcript excerpt, labelled as user supplied. Do not claim a quote has been verified against a video without evidence.
- The initial workflow uses student-entered timestamps. Automatic current-player timing, transcript fetching, speech transcription, and video understanding are later integrations.

Learning records (REC-1 through REC-3; REC-4 deferred)

- Create a record containing the assignment brief, answer, course, topic, code, commands, outputs, screenshots, and ordered steps explaining why each step was performed.
- Support drafts and an explicit finish action. Require a title, course, brief, and at least one explained step to mark a record finished.
- Copy imported evidence into app-managed storage, rather than depending on the original installation or file path. Retain provenance and distinguish student-entered evidence from captured execution output.
- Link related notes; search records and export an open-format bundle containing JSON, Markdown, and evidence files.
- Do not execute commands entered into a record.

- Provide SQLite text search now. Vector indexing, AI explanations, and revision-question generation depend on the deferred tutor/model infrastructure and are not part of this first release.

Style profiles (STY-1 through STY-5, with staged delivery)

- Start with a questionnaire for report, lab journal, or proposal profiles: cover-page fields, fonts, margins, headings, captions, spelling variant, references, voice, and code conventions.
- Save immutable profile versions. Proposed edits remain drafts until explicitly approved in the app; locked profiles cannot be changed until unlocked.
- Allow separate profiles per document type. An approved version is selected explicitly for each style check.
- Support local writing-sample attachment and basic measured sentence-length statistics. Automatic inference of formatting or writing voice from samples is a later enhancement, not a promised initial capability.
- Check pasted text for measurable deviations and clearly identify checks that cannot be assessed from plain text, including fonts, margins, and page layout.
- Resolve settings in this order: instructor requirements, selected approved profile, app defaults. Include the resolved settings in exports for a future document generator.
- Store the "match my style but fix errors" preference. Full document generation and error correction remain deferred until the generation module exists.

Explicitly deferred

Tutor chat, RAG/vector search, local language models, fine-tuning, flashcards, adaptive revision, video transcripts, mascot overlays, cloud providers, Privacy Gateway, lab installation/execution, generated PDF/DOCX reports, coding-agent integration, portal automation, and submission packaging. The single capture activation and comment popup are now included by the user's clarification.

Architecture baseline

Use Electron as the Windows desktop shell, with a React frontend, local FastAPI backend, SQLite metadata, and app-managed plain-file attachments. This user-directed decision supersedes the source requirements' Tauri choice. Keep notes, OCR, records, and styles behind separate service boundaries. The first integration slice can run in a browser against the same local API before Electron packaging.

The Electron main process owns desktop integration and the FastAPI child-process lifecycle. Expose only the required desktop operations through a narrow preload interface, with renderer context isolation enabled and Node integration disabled. Package the Python backend and local dependencies for Windows, and shut down the owned backend when the app exits. Measure the Electron shell's memory use on the reference laptop before setting a validated performance budget.

Keep study data in the student-selected workspace folder. Store only runtime settings, including the remembered workspace location, in the Windows per-user application-data directory. Bind the backend to loopback, validate allowed origins, and require a per-launch session token. Never send notes, evidence, or writing samples to a remote service in this release.

Use schema migrations, foreign-key enforcement, transaction boundaries, and crash-safe attachment writes. Imports receive generated storage names; user-supplied filenames never become filesystem paths. Store attachment hashes and sizes. Export must remain usable on a machine without the app.

Initial data model

Course: ID, name, timestamps

Topic: ID, course ID, name

Note: ID, title, body, comment, course/topic IDs, type, source app/time, optional source reference, timestamps

Tag / note tag: Unique tag name; note-to-tag relationship

Attachment: ID, generated relative storage path, original filename, media type, byte size, SHA-256, imported timestamp, provenance

Note attachment / record attachment: Explicit foreign-key relationships to attachments

OCR job: ID, attachment ID, state, provider, language, raw result, corrected result, error, timestamps

Learning record: ID, title, course/topic IDs, brief, answer, draft/finished status, finished timestamp

Record step: ID, record ID, sequence, action, rationale, command, output; optional evidence links

Record note: Record-to-note relationship

Video source: ID, original URL or owned local attachment ID, title, optional creator and duration in milliseconds

Video citation: ID, video source ID, start/end milliseconds, comment, optional user-supplied transcript excerpt

Note citation / record citation: Explicit note/record-to-video-citation relationships

Style profile: ID, name, document type, locked state, active approved version ID

Style version: ID, profile ID, version number, settings, draft/approved state, approval timestamp

Style sample: Profile-to-attachment relationship and extracted local text

Filing preference: Confirmed course/topic/type mapping and usage count

Action event: Entity ID, action, timestamp, relevant version/provenance information

Build order and completion checks

1. Foundation: repository setup, frontend/backend boundaries, migrations, local storage, and API session protection. Verify database persistence across restart and rejection of unauthorized requests.
2. Notes vertical slice: create, edit, list, filter, attach, and export. Verify note content and attachment bytes survive restart and export retains source metadata.
3. OCR integration: queued extraction, review, correction, and retry. Verify a known printed-text fixture, an invalid image, unavailable OCR engine, and preservation of the original image on failure. Measure accuracy on representative course screenshots before choosing the final engine.
4. Learning records: explained steps, owned evidence, note links, finish validation, search, and export. Verify records remain readable after the original imported evidence is removed and export includes every referenced attachment.
5. Style profiles: questionnaire, drafts, approval, immutable versions, locking, instructor overrides, and limited consistency checks. Verify draft edits do not change the active approved profile, locked profiles reject changes, and instructor values override profile values.

6. Windows packaging and offline demo: package the shell/backend and document the local OCR prerequisite or bundled distribution. Test all four workflows with networking disabled and verify no external requests occur. Test backup restoration on a fresh data directory.

Video citations are part of the notes and records slices: verify timestamp parsing and exact millisecond round trips, multiple moments per video, invalid range rejection, provider-link generation, generic URL fallback, and portable exports. Verify saved remote citations remain readable offline and local citations retain their video evidence after the original file is removed. Do not fetch remote URLs during citation creation.

The release is complete when these implemented workflows pass their checks on Windows and the UI accurately describes deferred capabilities. No automatic AI feature should be represented by a placeholder success result.

Historical requirements

Original document dated 2 October 2026

The following eleven pages are preserved unchanged. The Electron choice and focused Phase 1 scope in the revision at the beginning supersede their conflicting decisions. All other future-phase requirements remain reference material.

Scholo Education Assistant: Project Requirements Document

Oct 2, 2026 · @Ahmad Jahangir

1. Overview and Goals

Scholo (working name) is a local-first desktop study companion for CS students that explains course material, captures what is on screen, helps prepare assignments in a sandboxed lab, and turns every finished assignment into reusable exam knowledge.

Problem. Students lose time on assignment setup, screenshots, formatting and scattered notes, and the work they finish is rarely kept in a form they can revise from later.

Goals

- Tutor from the student's own slides, PDFs and notes, with answers cited to their source.
- Capture anything on screen with one shortcut, understand it, and file it where the student wants.
- Set up the software an assignment needs in an isolated space, run it for real, capture the evidence, and build the journal, report or zip.
- Keep every finished assignment as a learning record that the AI can later explain and quiz from.
- Keep the student's cover page, tone and formatting consistent across all assignments.
- Run fully offline on a student laptop, with optional free cloud models for heavy tasks.
- Serve as an AI course project: a locally fine-tuned model, a computer vision component and a measured evaluation.

Non-goals

- Copying or reusing another student's work. AI assistance is allowed; plagiarism is not, and the product helps keep work unique.
- Submitting anything without the student's explicit approval.
- Paid infrastructure, virtual machines or a hosted multi-user service in the first version.
- Training a language model from scratch.

2. Users, Platform and Target Environment

The primary user is a university CS student working alone on a Windows laptop, and the product is designed for the weakest realistic machine rather than a workstation.

- **Platform:** Windows 10/11 desktop app. Tauri shell, React frontend, local FastAPI backend.
- **Storage:** SQLite for records and metadata, a local vector store for retrieval, plain folders for files.
- **Reference hardware:** Intel Core i5-10210U (4 cores), 16 GB RAM, Intel UHD integrated graphics, no GPU, about 190 GB free disk.
- **Lite mode:** 8 GB RAM machines use a 1.5B model and reduced background indexing.
- **Network:** everything required for the core experience works offline; the cloud is optional.

3. Scope and Delivery Tiers

The project is built in three tiers, each usable and demonstrable on its own, with security and testing running across all of them.

Tier	Contents	Depends on
1. Core	Tutor and RAG, Point and Ask, Smart Capture Notes, OCR, learning records, style profile, mascot overlay, local model	None
2. Lab	Sandboxed environments, evidence capture, report builder, submission packaging, coding agent integration	Tier 1
3. University automation	Portal integration, assignment detection and download, analysis and generation, validation, approval, upload step	Tiers 1 and 2
Optional cloud boost	Free LLM providers for large tasks through the Privacy Gateway	Tier 1

4. Functional Requirements

Requirements are grouped by module and tagged with their tier (T1, T2, T3).

Tutor, RAG and adaptive learning (T1)

- TUT-1: Index local files (PDF, slides, notes, images, code) into a searchable knowledge base.
- TUT-2: Answer questions from that knowledge base with citations to file and page.
- TUT-3: Track weak topics from quiz results and learning records, and schedule revision.
- TUT-4: Generate summaries, flashcards and practice questions per course topic.

Point and Ask capture (T1)

- CAP-1: A global shortcut makes the mascot grow and dims the rest of the screen slightly. It is not a plain snipping tool.
- CAP-2: Clicking anywhere identifies the content type: video, PDF, slide, text, diagram, code, table or other. Window metadata and OCR do most of the work, with the UI detector as support.
- CAP-3: A drag selection overrides the guess when the click target is wrong.
- CAP-4: The student then says what to do, by text or voice: save, make notes, explain, cite later, or add to flashcards.
- CAP-5: For video, store the page, URL, timestamp and transcript when available, plus the student's comment, so later answers can cite that video and moment. The local model works from the transcript and notes, not the video itself.
- CAP-6: For PDFs and slides, record the file and page, extract the selected region, and add it to a notes document with a source reference.
- CAP-7: When capture returns a black frame (protected content), fall back to metadata and transcript.

Smart Capture Notes (T1)

- NOT-1: After any capture, a small box accepts comments, tags and details.
- NOT-2: The helper sorts the item by course, topic and type, and shows where it will be filed. The student can change the destination, and the helper learns the preference.
- NOT-3: Each capture stores the image, comment, OCR text, source app and timestamp.

Learning records (T1)

- REC-1: Every finished assignment is saved with its brief, answer, code, commands, outputs, screenshots and a step-by-step explanation of why each step was done.
- REC-2: Records are tagged by course and topic and are indexed into the knowledge base.
- REC-3: Records survive deletion of the software used to produce them, since evidence is stored, not installations.
- REC-4: The AI can explain how any past assignment was done and generate revision questions from it.

Personal style and consistency (T1)

- STY-1: Build a style profile from the student's past work: cover page templates, fonts, margins, headings, captions, writing voice, spelling variant, referencing style and code conventions.
- STY-2: Apply the profile to every generated document, and let the student keep separate profiles (report, lab journal, proposal) and lock one.
- STY-3: Produce a consistency report on each draft that highlights deviations in sentence length, vocabulary and formatting.
- STY-4: Instructor requirements override the profile. A short questionnaire covers the cold start. Profile changes are versioned and need approval.
- STY-5: Offer a "match my style but fix errors" option.

Lab environment and evidence capture (T2)

- LAB-1: Detect the software an assignment needs from its brief.
- LAB-2: Use the lightest isolation that works: Python venv or conda and per-assignment folders first, portable user-level installs second, Docker and Windows Sandbox only as optional extras. No virtual machines.
- LAB-3: Install only from an allowlist, ask before each install, enforce a disk quota, and require explicit confirmation for any system-wide installer.
- LAB-4: Keep a manifest of everything installed during an assignment.
- LAB-5: When the assignment is done, list installed items with sizes and ask: delete all, delete some, or keep. Act on the answer and log it.
- LAB-6: Run the real code or steps. Outputs and screenshots come only from actual execution and are timestamped and linked to the run that produced them.

- LAB-7: Capture screenshots when the UI detector sees an output or error appear. If automation fails, fall back to guided manual capture.

Assignments, validation and approval (T3, with local parts in T2)

- ASG-1: Detect new assignments from the university portal and download their briefs and files.
- ASG-2: Analyze the brief and produce a plan and draft. AI generation is allowed, and drafts are made unique to the student.
- ASG-3: Run an originality check against the student's own past work and an optional web-similarity check.
- ASG-4: Validate drafts by re-running code, checking the brief's requirements and checking format.
- ASG-5: The student reviews, edits and approves every deliverable before it is packaged.
- ASG-6: Feedback on past drafts feeds personalization.

Submission packaging (T2/T3)

- SUB-1: Each assignment has a submission profile: required files, naming, folder structure and format (zip, PDF, DOCX or portal form).
- SUB-2: Build the folder structure and zip (for example code, report, screenshots, README) to match the profile.
- SUB-3: Run a pre-submission checklist: file names and sizes, a clean run of the zipped code, and a match between report and brief.
- SUB-4: Upload to the portal only after an explicit click by the student.

University portal (T3)

- POR-1: Prefer official APIs (Canvas, Moodle or similar) where the university provides them.
- POR-2: Otherwise the student logs in themselves and the app only reads.
- POR-3: Store credentials in the Windows Credential Manager and never send them to any model.

Coding agent integration (T2)

- AGT-1: Hand build tasks to a coding agent such as Antigravity, supplying the brief, course notes and constraints.
- AGT-2: Bring results, outputs and explanations back into the learning record. Code sent to a hosted agent follows the Privacy Gateway rules.

Mascot and cursor companion (T1)

- UI-1: A small always-on-top overlay follows the pointer with three states: idle, capturing and working.
- UI-2: It can be turned off and animates only on events to keep CPU use low.

5. AI and ML Requirements

Small quantized models run locally by default, training happens once on free cloud GPUs, and large tasks may go to free LLM providers through the Privacy Gateway.

Component	Choice	Runs where
Language model	1.5B to 3B parameters, 4-bit quantized, via Ollama or llama.cpp	Local
Fine-tuning	LoRA or QLoRA on a domain dataset (explanations, assignment structure, lab reports), exported as adapter or GGUF	Colab or Kaggle, then local
Embeddings	Small model (bge-small or MiniLM class)	Local
OCR	Tesseract or PaddleOCR	Local
UI detector	YOLO-nano class model fine-tuned on screenshots of IDEs, terminals and common tools	Trained on Colab, runs local
Classifiers	Assignment type and topic tagger	Local
Style adapter	Optional LoRA for personal writing style	Trained on Colab, runs local

Component	Choice	Runs where
Large-task models	Free LLM providers via a router (9Router, OmniRoute) or the Gemini free tier	Cloud, only through the Privacy Gateway

Routing rules

1. Simple and private tasks go to the local model.
2. Large or complex tasks (long reports, big code generation) may go to free providers after redaction.
3. If a provider fails or is not allowed, fall back to the local model.
4. The full demo must run offline.

Evaluation. Report tutoring answer accuracy, retrieval hit rate, OCR accuracy, detector mAP, redaction leak rate and restoration accuracy, style-match score on held-out assignments, and user ratings, each with and without the relevant component.

6. Privacy, Security and Permissions

Nothing leaves the machine unless it has passed the Privacy Gateway and the student has allowed that feature to use the cloud.

Privacy Gateway

1. Detect sensitive items (names, student IDs, emails, phone numbers, course and university identifiers, paths, usernames) with regex plus a small NER model.
2. Replace each with a placeholder and keep the mapping in a local, encrypted, per-request vault.
3. Send only the redacted prompt.
4. On the response, verify the placeholders survived, restore the originals from the vault, then discard the mapping.

Rules

- Hard block: passwords, tokens, cookies and API keys are never sent, even as placeholders. The request goes to the local model instead.
- Fail closed: if detection is uncertain, route locally or show a preview and ask.
- Screenshots are OCR'd first and sensitive regions redacted before any image-derived content is sent.

- A non-sensitive technical terms allowlist stops the gateway from breaking tasks.
- Writing samples used for the style profile are treated as sensitive.
- Each cloud use is opt-in per feature and clearly marked local or cloud.

System controls

- Credentials live in the Windows Credential Manager.
- Any router runs on localhost only, requires an API key, uses a pinned version and has unused providers disabled. Provider terms are checked before use, and account pooling is avoided.
- Installs follow the allowlist, prompt and disk quota in LAB-3.
- An action log records installs, deletions, cloud requests and approvals.

7. Non-Functional Requirements

The app must stay responsive on a 4-core, 16 GB, CPU-only laptop. The figures below are targets to validate during testing.

- **Performance:** the capture overlay appears in about 1 second; a clicked item is identified in about 3 seconds; model output streams as it is generated, since a 3B model on CPU produces only a few tokens per second.
- **Memory budget:** about 0.5 GB for the OS-level app shell, about 2.5 GB for the language model, and about 1 GB for embeddings and OCR, leaving headroom on 16 GB.
- **Offline:** all Tier 1 features work with no network.
- **Reliability:** cloud failures degrade to the local model; interrupted installs and runs can be resumed or cleaned up from the manifest.
- **Usability:** every action that changes files, installs software or contacts the cloud is visible and reversible where possible.
- **Maintainability:** modules sit behind clean API boundaries (model provider, capture, lab, portal) so providers and tools can be swapped.
- **Portability of data:** records and notes can be exported and backed up in open formats.

8. Constraints, Assumptions and Risks

The design assumes free tools only, a CPU-only reference machine and a student who stays in the loop; the risks below remain after mitigation.

Assumptions

- AI assistance is permitted on the student's courses; plagiarism is the only prohibited use.
- Free Colab or Kaggle GPU sessions are enough for LoRA and detector training.
- The student's university offers either an API or a manual login the app can use read-only.

Risk	Mitigation
Redaction is never perfect	Layered detection, hard-block list, fail closed, local-only default for sensitive material, measured leak rate
Free cloud tiers change or disappear	Provider abstraction with local fallback; check current limits and terms before relying on any provider
Free tiers may prohibit relaying	Read each provider's terms; avoid account pooling
Plagiarism or similarity flags	Originality checker against the student's own work and optional web check; unique structure and wording
A 3B model struggles with long or complex tasks	RAG instead of long context; route heavy tasks to free providers
GUI automation is fragile	Capture on detected events, scripted steps for common tools, manual fallback
Installs damage the system	Lightest isolation first, allowlist, manifest, quota, confirmation for system installers
Portal changes or blocks access	Official APIs first; read-only student-initiated login; manual submission
Scope is large	Three tiers, each demoable alone
Name may already be taken	Treat Scholo as a working name until trademark, domain and store checks are done

9. Deliverables Map

The original 26 deliverables plus 10 additions from this planning phase make 36 in total.

Group	Deliverables
Planning (current stage)	1 Requirements, 2 Architecture diagram, 3 Technology stack, 4 UI/UX and wireframes, 5 Database and ER design, 20 Security and permissions (design)
Core build	6 Frontend, 7 FastAPI backend, 8 AI study assistant, 9 Adaptive learning, 10 Knowledge base and RAG, 11 File and screenshot management, 12 OCR and shortcut
University automation	13 Portal integration, 14 Assignment detection and download, 15 Analysis and generation, 16 Validation and testing, 17 Review and approval, 18 Feedback and personalization, 19 Upload and submission
Finish	21 Testing and QA, 22 User documentation, 23 Developer documentation, 24 Final application, 25 Final report, 26 Demo and presentation
Added	27 Lab Environment and Evidence Capture, 28 Learning Records, 29 Submission Packaging, 30 Local Model Training and Evaluation, 31 Computer Vision, 32 Privacy Gateway, 33 Smart Capture Notes, 34 Point and Ask Mode, 35 Coding Agent Integration, 36 Personal Style and Consistency

10. Open Questions and Next Steps

Seven questions were answered on 2 October 2026, and the decisions below are now fixed; the remaining open items follow the table.

Question	Decision	Effect
Windows edition	Windows 11 Pro	Windows Sandbox stays available as an optional isolation tier in LAB-2
App shell	Tauri	Lighter app; native code needs some Rust, so most logic stays in the Python backend

Question	Decision	Effect
University portal	Bahria CMS (cms.bahria.edu.pk), no API	The student logs in and the app only reads (POR-2); there is no official API route
Antigravity integration	Command line or IDE driving	The exact route is confirmed against Antigravity's current capabilities before AGT-1 is designed
Free cloud providers	Those offered through OmniRoute	Each provider is switched on individually after its terms are read; all traffic passes the Privacy Gateway
Product name	Not decided	Scholo stays a working name
Submission formats	PDF and zip	The first two submission profiles for SUB-1

Still open

- Final product name, plus trademark and domain checks.
- The exact Antigravity interface (command line options or IDE driving method).
- Per-provider terms and free-tier limits for the cloud providers chosen.
- Technical choices tracked in the Technology Stack document: vector store, OCR engine and base model.

Next planning documents: UI/UX wireframes (#4) and database design (#5); the architecture overview (#2) and technology stack (#3) are done.